

ENCORE

Ticket Booking Platform
System Design Write-Up

Ajitesh Sharma

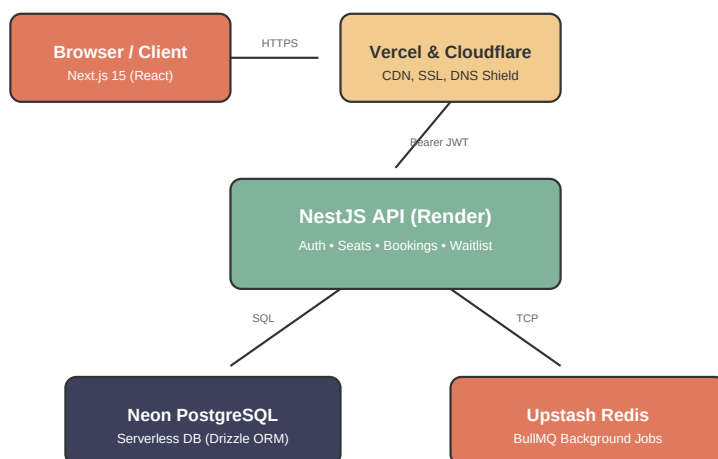
VIT Vellore · Batch 2023 · Passout 2027

Computer Science & Engineering — Information Security

1. Architecture Overview

The system is split into three independently deployed tiers. The frontend is a Next.js 15 app on **Vercel's global edge CDN**. The backend is a NestJS (Node.js) REST API on **Render** that handles auth, seat management, bookings, payments, and background jobs. State is stored in a **Neon serverless PostgreSQL** database, and background jobs are coordinated via **BullMQ backed by Upstash Redis** (serverless, no always-on cost). The domain **tickets.ajiteshsharma.dev** is managed on **Cloudflare** (DNS-only, grey cloud) for DDoS protection and fast DNS resolution, with Vercel issuing and renewing SSL automatically.

Figure 1 — Full deployment architecture.

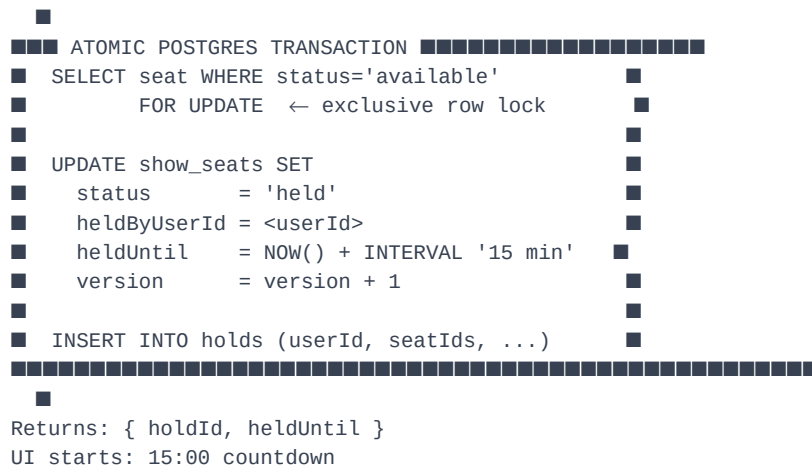


2. Seat Hold & TTL Mechanism

When a user selects seats and moves to checkout, those seats need to be temporarily reserved so nobody else can buy them mid-payment. But they can't be locked forever if the user just closes the tab. The solution is a **15-minute TTL (Time-To-Live) hold**.

The moment the user clicks 'Hold Seats', a single **atomic Postgres transaction** fires: it reads the seat, verifies it is still available, then writes the new state — `status = held`, `heldByUserId`, and critically **`heldUntil = NOW() + 15 minutes`**. That timestamp is the TTL. From this point every downstream API call — generating a payment intent, confirming the booking — checks the server clock against **`heldUntil`**. If the TTL has passed, the API returns **HTTP 409 Conflict** immediately and the cart is dead. No periodic sweep needed; the TTL enforces itself inline.

```
POST /api/shows/:showId/hold
```



```
FOR UPDATE SKIP LOCKED    (safe multi-worker)
■
Found next user in queue?
YES ■■■ Assign new 15-min hold directly to user
    Mark entry → 'offered', set offerExpiresAt
    Send email via Resend: '15 min to claim!'
NO  ■■■ Seat returns to general public pool
```

Figure 3 — Waitlist auto-allocation triggered by hold expiry.

5. Time-Limited Offer Handling

Once offered a seat, the waitlisted user has exactly **15 minutes** — enforced by the same TTL engine as a regular hold. If they ignore the email and the offer expires, the BullMQ worker's next cycle detects `offerExpiresAt <= NOW()`, marks the entry *expired*, releases the hold, and immediately runs allocation again for the next person in line. This creates a self-sustaining cascade — tickets flow down the waitlist automatically with no manual intervention from the event organiser.

Worker scheduling is self-rescheduling — no external cron needed:

- App starts → inserts first **release_expired_holds** job into Postgres.
- BullMQ pulls the job via Upstash Redis and executes it.
- After completion → schedules next run 30 seconds later.
- Cycle repeats indefinitely for the lifetime of the process.

6. Security

Security decisions are consistent throughout the stack — no custom crypto, no session databases, no plaintext secrets.

Mechanism	Implementation	Effect
JWT	HS256, 7-day access + refresh rotation	Stateless auth, no session store
Passwords	Argon2id (memory-hard)	GPU brute-force resistant
DNS	Cloudflare — DDoS shield, fast resolution	Protects origin IP
CORS	Whitelist: Vercel + ajiteshsharma.dev	Blocks rogue cross-origin calls
Rate limit	Throttler: 100 req / 60s / IP	Stops brute-force login
SQL	Drizzle ORM parameterised queries	No injection risk
Headers	Helmet middleware on all responses	XSS, clickjack, MIME protection